C# Roadmap: Merged 12-Week Plan

October 9 to December 31, 2026: an intensive path from complete beginner to a deployed ASP.NET Core Web API portfolio project

Total duration	Weekly commitment	Primary target
12 weeks (84 days)	18 - 22 hours (core track: 10 - 12)	Deployed Web API on GitHub

This plan combines the two earlier roadmaps. It uses the faster pacing of the accelerated plan (web API work starts in Week 9, with a full week for polish at the end) and keeps the extras from the 12-week plan: SQL basics, generics, pattern matching, validation, Docker deployment, a first-week day-by-day guide, a fallback plan and the expert path for 2027.

Two tracks, one calendar. Every week lists **Core** items and **Stretch** items. With 18 - 22 hours a week, do both. If you can only manage about 10 - 12 hours, do the Core items only; the dates and final project stay the same. Only the stretch items drop away.

A realistic expectation: in 12 weeks you can become a confident junior to early-intermediate C# developer with a real project to show. Expert level (architecture, performance tuning, runtime internals) takes years of practice. Phase 7 outlines how to continue after December 31.

The journey at a glance

Phase	Focus	Weeks	Dates
1	Setup and C# fundamentals	1 - 2	Oct 9 - Oct 22
2	Object-oriented programming	3 - 4	Oct 23 - Nov 5
3	Intermediate C# and data handling	5 - 6	Nov 6 - Nov 19
4	Advanced C# and runtime concepts	7 - 8	Nov 20 - Dec 3
5	Web API and databases	9 - 11	Dec 4 - Dec 24
6	Portfolio polish and delivery	12	Dec 25 - Dec 31
7	Expert path (continues in 2027)	-	Jan 2027 onward

Execution rules

- **Time:** aim for 2 - 3 hours on weekdays and 4 - 5 hours on weekends. Consistency creates momentum.
- **70/30 ratio:** 30% studying concepts, 70% writing code and debugging.
- **Daily commits:** commit progress to GitHub every day as proof of work and as your portfolio.
- **Type everything yourself.** Never copy-paste from tutorials.
- **Time-box being stuck:** after 30 - 45 minutes, search, ask, or move on and return later.
- **Protect the final project.** If you fall behind, drop stretch items, never the Web API.

Phase 1: Setup and C# Fundamentals

Weeks 1-2 | Oct 9 - Oct 22 | Goal: master the CLI workflow, control flow, methods and basic collections

Week 1 (Oct 9 - Oct 15): Setup and Core Syntax

Core: install the .NET SDK and an editor (Visual Studio, VS Code with C# Dev Kit, or Rider); dotnet CLI (new, build, run); Git and GitHub basics; variables and primitive types; type conversion; string interpolation; operators; if/else; switch expressions; for, foreach and while loops.

Stretch: interactive text-based quiz game.

Build: number-guessing game and a calculator with a menu.

Week 2 (Oct 16 - Oct 22): Methods, Collections and Debugging

Core: methods (parameters, return values, overloading); arrays and `List<T>`; string handling; exceptions (try/catch/finally); nullability operators; debugger breakpoints and stepping.

Stretch: unit converter.

Build: console to-do list, structured cleanly and committed to GitHub.

> **Checkpoint:** you can solve beginner challenges (FizzBuzz, palindromes, simple sorting) without looking up the answer, and you can create, run and commit a project unaided.

Phase 2: Object-Oriented Programming

Weeks 3-4 | Oct 23 - Nov 5 | Goal: model real-world domains cleanly with OOP design principles

Week 3 (Oct 23 - Oct 29): Encapsulation and Abstraction

Core: classes, objects, fields, properties, constructors; access modifiers (private, public, internal); static members; namespaces; clean-code habits (naming, small methods, single responsibility).

Build: bank account system (accounts, transactions, validation, error handling).

Week 4 (Oct 30 - Nov 5): Inheritance, Polymorphism and Interfaces

Core: inheritance; virtual/override; polymorphism; interfaces vs abstract classes; records; enums; value vs reference types.

Stretch: sealed classes, struct basics.

Build: inventory or library management system built on interface abstractions.

> **Checkpoint:** you can sketch a small class design before coding and explain why you chose an interface or inheritance.

Phase 3: Intermediate C# and Data Handling

Weeks 5-6 | Nov 6 - Nov 19 | Goal: write idiomatic C# with LINQ, collections and unit tests

Week 5 (Nov 6 - Nov 12): Collections, LINQ and File I/O

Core: Dictionary and HashSet; generics; LINQ (Where, Select, GroupBy, OrderBy, Join); File I/O; JSON serialization with System.Text.Json.

Stretch: Queue and Stack; generic constraints.

Build: JSON expense tracker with LINQ-generated reports.

Week 6 (Nov 13 - Nov 19): Delegates, Lambdas and Unit Testing

Core: Func and Action; lambdas; extension methods; pattern matching; unit testing with xUnit using the Arrange-Act-Assert pattern.

Stretch: events; CSV/JSON data analyzer.

Build: full xUnit test suite for your expense tracker or bank system.

> **Checkpoint:** you reach for LINQ naturally and your code has automated tests.

Phase 4: Advanced C# and Runtime Concepts

Weeks 7-8 | Nov 20 - Dec 3 | Goal: master async code, memory basics and dependency injection

Week 7 (Nov 20 - Nov 26): Asynchronous Programming and Memory

Core: async/await; Task; CancellationToken; common async pitfalls; IDisposable and using statements; stack vs heap; garbage collection (conceptual).

Stretch: threading basics, reflection, Span<T>.

Build: async web/data downloader with progress tracking and cancellation.

Week 8 (Nov 27 - Dec 3): Dependency Injection and SOLID

Core: dependency injection and inversion of control; SOLID principles; Repository, Factory and Strategy patterns.

Stretch: Observer pattern.

Build: refactor an earlier console application to use DI and SOLID.

> **Checkpoint:** you can explain how async/await works and justify your design choices.

Phase 5: Web API and Databases

Weeks 9-11 | Dec 4 - Dec 24 | Goal: build a production-style Web API backend

Week 9 (Dec 4 - Dec 10): ASP.NET Core Fundamentals

Core: RESTful API design; controllers vs minimal APIs; routing; model binding; middleware pipeline; validation and error handling.

Build: in-memory CRUD API (e.g., tasks or products).

Week 10 (Dec 11 - Dec 17): Databases and Entity Framework Core

Core: SQL basics (SELECT, INSERT, UPDATE, DELETE, JOIN); EF Core; DbContext; migrations; relationships; CRUD and relational querying. Use SQLite while developing.

Stretch: PostgreSQL or SQL Server locally.

Build: connect your API to a real database.

Week 11 (Dec 18 - Dec 24): Authentication and Deployment

Core: JWT authentication and authorization; OpenAPI/Swagger documentation; Docker basics.

Stretch: deploy to a free or low-cost cloud host.

Build: complete REST API backend (task management or e-commerce) with EF Core, JWT auth and Swagger.

> **Checkpoint:** your API supports full CRUD, authentication and Swagger docs, and runs against a real database.

Phase 6: Portfolio Polish and Delivery

Week 12 | Dec 25 - Dec 31 | Goal: prepare the repository for technical review and public presentation

Week 12 (Dec 25 - Dec 31): Integration, Testing and Documentation

Core: integration testing; basic performance tuning; structured logging; writing a clear GitHub README with setup steps and design decisions.

Stretch: CI pipeline with GitHub Actions.

Build: publish the full Web API on GitHub with architecture notes, tests and deployment setup (Docker).

Checkpoint: one public, working, documented project that you can show in interviews. Prefer desktop, games or mobile? Swap Phase 5 for WPF, Unity or .NET MAUI and keep the same capstone idea.

Phase 7: Expert Path (January 2027 onward)

Ongoing | Goal: turn a strong foundation into real expertise

What comes next

- **Architecture:** Clean Architecture, DDD, CQRS, microservices vs modular monoliths.
- **Performance:** profiling, BenchmarkDotNet, allocation analysis, caching.
- **Quality:** integration tests, observability, security best practices.
- **Runtime internals:** JIT, GC modes, how async and LINQ compile.
- **Cloud and DevOps:** CI/CD, Azure or AWS, infrastructure as code.
- **Craft:** system design, code review, mentoring, open-source contributions.
- **Practice:** build a second, more complex system and apply for junior or intern roles.

Checkpoint: you make sound design decisions and can diagnose problems in a real production system. This usually takes 1 to 3 years of practice.

Recommended resources

Type	Resource	Best for
Official docs	Microsoft Learn and the C# documentation	Free, accurate, up to date; use throughout
Practice	Exercism (C# track), LeetCode, Advent of Code	Problem solving (Phases 1-3)
Book	C# in Depth by Jon Skeet	Deep language understanding (Phase 3 onward)
Book	CLR via C# by Jeffrey Richter	How the runtime works (Phase 7)
Book	Clean Code and Clean Architecture by Robert C. Martin	Design principles (Phases 4-7)
Community	Official .NET blog, GitHub dotnet repos, r/csharp	Staying current and getting help

If you fall behind

- **Cut stretch items first.** They are labeled in every week.
- **Never skip fundamentals or OOP.** Everything later depends on them.
- **Shrink Phase 4** to async/await and DI; patterns can wait until January.
- **Simplify the Web API** (fewer endpoints, skip Docker) rather than abandoning it.
- **Use weekends to catch up.** There are no spare weeks in this schedule.

Your first 7 days (Oct 9 - Oct 15)

- **Fri Oct 9:** install .NET and your editor; run Hello World.
- **Sat Oct 10:** set up Git and GitHub; variables, types and user input.
- **Sun Oct 11:** operators, if/else and switch expressions with small exercises.
- **Mon Oct 12:** loops; practice with FizzBuzz and a multiplication table.
- **Tue Oct 13:** build the number-guessing game.
- **Wed Oct 14:** build the calculator with a menu.
- **Thu Oct 15:** review, note what confused you, commit everything and plan Week 2.

Check Microsoft Learn for the latest C# and .NET versions and any newer learning paths, as the ecosystem evolves quickly.